# Manual CSS Grid

https://johnserrano.co/blog/css-grid-la-guia-completa-para-principiantes-y-avanzados

CSS Grid Layout (o simplemente CSS Grid) es un sistema de diseño bidimensional que ha cambiado por completo la forma en que creamos interfaces web.

Durante años, CSS nos ha permitido maquetar páginas, pero siempre con soluciones improvisadas: primero con tablas, luego con flotados (floats), posicionamiento inline-block.

Ninguno de estos métodos fue diseñado específicamente para la maquetación y, por eso, tenían muchas limitaciones (como la falta de alineación vertical).

Flexbox es un gran avance en diseño web, pero su enfoque unidimensional hace que no siempre sea la mejor opción. Aquí es donde CSS Grid brilla: es el primer módulo creado con el propósito exclusivo de resolver los problemas de diseño web, permitiéndonos estructurar nuestras interfaces de manera más intuitiva y eficiente.

En esta guía, exploraremos los conceptos clave de Grid según la versión más reciente de la especificación. No abordaremos la sintaxis obsoleta de Internet Explorer, aunque Grid sigue siendo compatible con IE 11.

## Aprende CSS Grid con ejemplos prácticos

Si quieres ver CSS Grid en acción y descubrir cómo crear diseños responsivos de forma sencilla, aquí tienes un video donde explico paso a paso cómo estructurar tus cuadrículas y aplicar buenas prácticas. 👇

https://www.youtube.com/watch?v=8REXun8U5OY

## Conceptos básicos de CSS Grid

Desde marzo de 2017, la mayoría de los navegadores incorporaron soporte nativo y sin prefijos para CSS Grid, incluyendo Chrome, Firefox, Safari (también en iOS) y Opera.

Por otro lado, Internet Explorer 10 y 11 lo admiten, pero con una implementación antigua y una sintaxis obsoleta. ¡El momento de empezar a usar Grid es ahora!

Para comenzar, lo primero es definir un contenedor como una cuadrícula con `display: grid`. Luego, se establecen las dimensiones de las filas y columnas con `grid-template-columns` y `grid-template-rows`. Finalmente, se ubican los elementos hijos dentro de la cuadrícula con `grid-column` y `grid-row`.

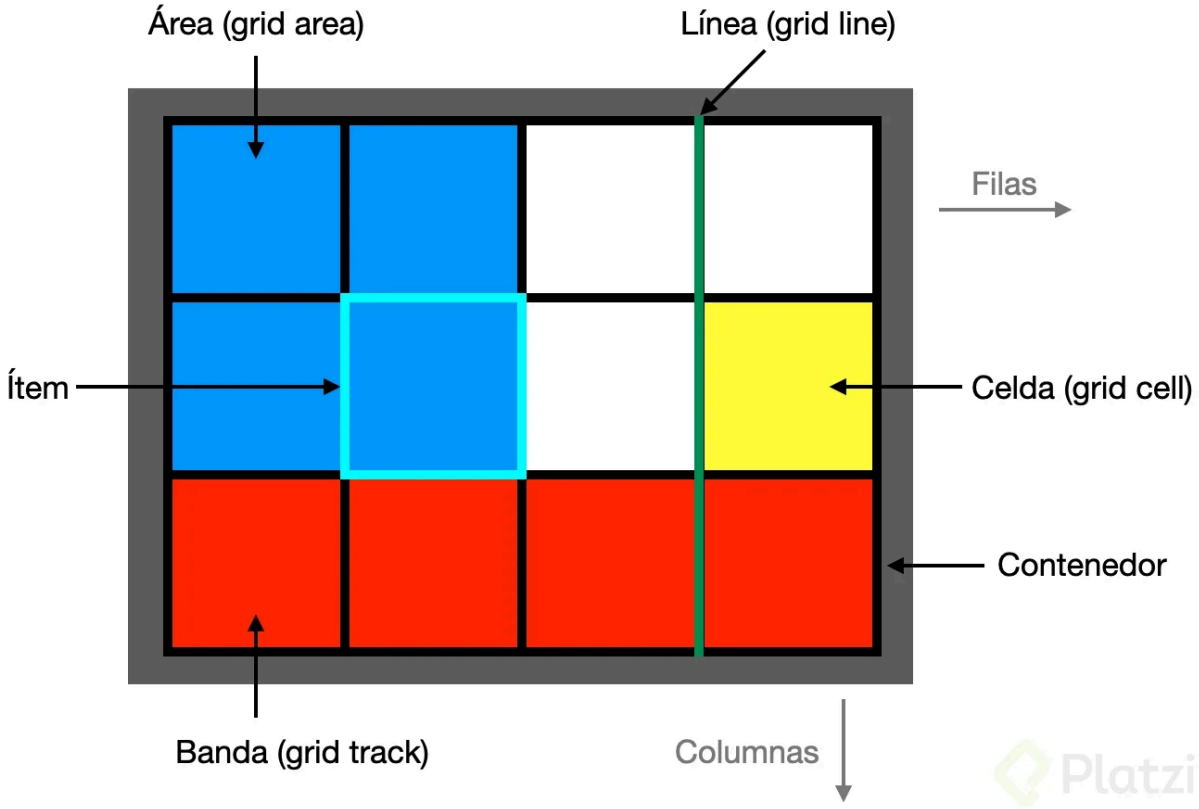
Al igual que Flexbox, el orden en el código fuente no afecta la posición de los elementos en la cuadrícula.

Esto significa que puedes reordenar tu diseño fácilmente con media queries, adaptándolo a distintos tamaños de pantalla con solo unas pocas líneas de CSS.

Imagina definir toda la estructura de una página y luego reorganizarla completamente para dispositivos móviles sin modificar el HTML.

CSS Grid es uno de los módulos más potentes que se han incorporado a CSS, brindando flexibilidad y control total sobre el diseño de nuestras interfaces.

Te dejo una gráfica para que sea más visual:




### Términos clave en CSS Grid

**Contenedor**: El elemento padre que define la cuadrícula.

**Ítem**: Cada elemento hijo dentro de la cuadrícula.

**Línea (grid line)**: Las líneas horizontales y verticales que separan las celdas.

**Celda (grid cell)**: La unidad mínima de la cuadrícula, equivalente a un solo “cuadro”.

**Banda (grid track)**: Una fila o columna completa dentro de la cuadrícula.

**Área (grid area)**: Un conjunto de celdas que forman una región dentro de la cuadrícula.

### Grid Container

Es el elemento al que se le aplica `display: grid;` y actúa como el contenedor principal de todos los elementos de la cuadrícula. En este ejemplo, el contenedor es el grid container.

```
<div class="contenedor">
  <div class="item item-1"> </div>
  <div class="item item-2"> </div>
  <div class="item item-3"> </div>
</div>
```

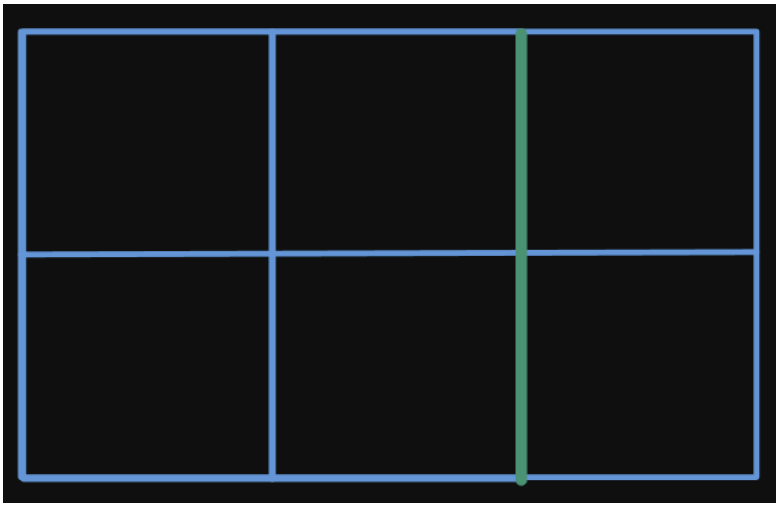
### Grid Item

Son los elementos hijos directos del contenedor de la cuadrícula. En este ejemplo, los elementos item son `grid items`, pero sub-item no lo es.

```
<div class="contenedor">
  <div class="item"> </div>
  <div class="item">
    <p class="sub-item"> </p>
  </div>
  <div class="item"> </div>
</div>
```

### Grid Line

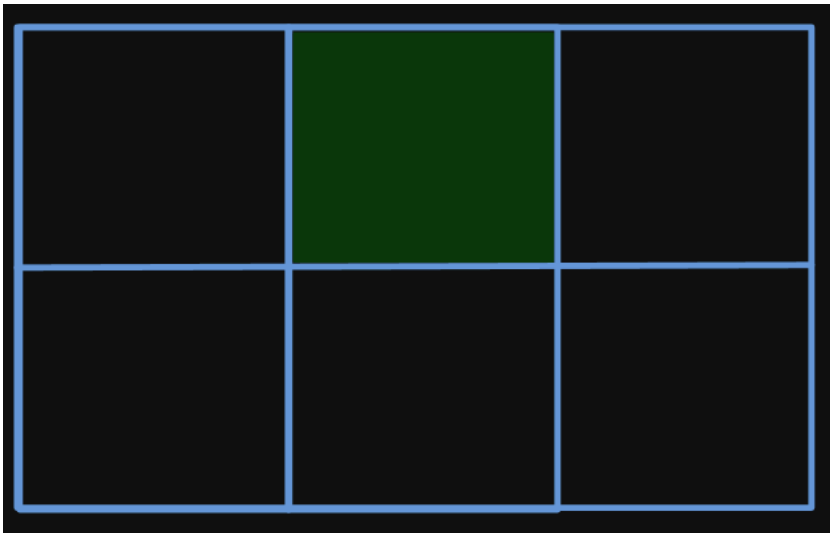
Son las líneas divisorias que conforman la estructura de la cuadrícula. Pueden ser verticales (líneas de columna) o horizontales (líneas de fila) y se encuentran a ambos lados de una fila o columna. En este ejemplo, la línea verde



representa una línea de columna.

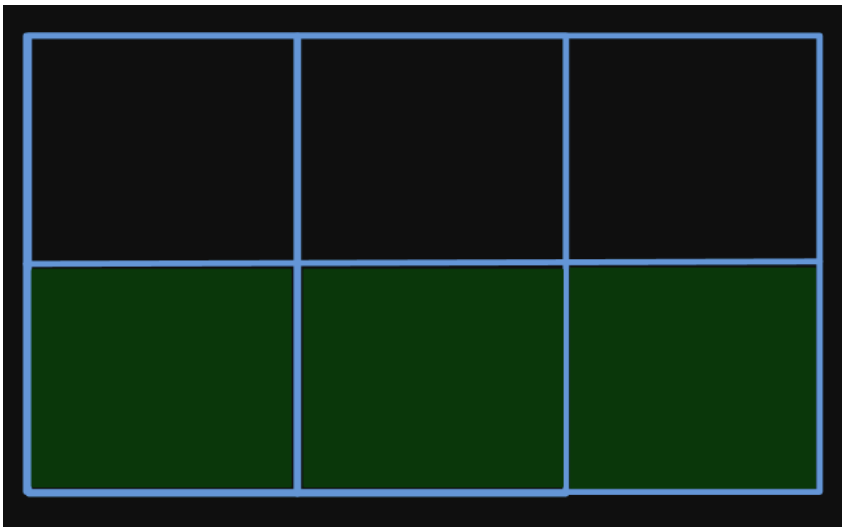
### Grid Cell

Es el espacio comprendido entre dos líneas de fila y dos líneas de columna adyacentes. Representa la unidad más pequeña dentro de la cuadrícula. En este ejemplo, la `grid cell` está ubicada entre las líneas de fila 1 y 2 y las líneas de columna 2 y 3.



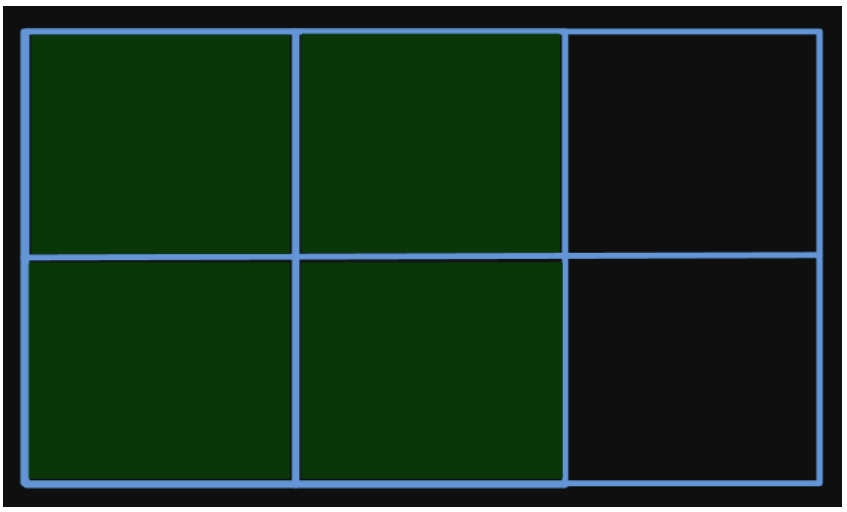
### Grid Track

Es el espacio comprendido entre dos líneas de la cuadrícula adyacentes. Puede representar una fila o una columna dentro de la cuadrícula. En este ejemplo, el `grid track` se encuentra entre la segunda y tercera línea de fila.



### Grid Area

Es el espacio total delimitado por cuatro líneas de la cuadrícula. Puede estar compuesto por una o varias `grid cells`. En este ejemplo, el grid area se encuentra entre las



líneas de fila 1 y 3 y las líneas de columna 1 y 3.

## Propiedades de CSS Grid

CSS Grid se compone de una serie de propiedades que se aplican tanto al contenedor de la cuadrícula (grid container) como a los elementos de la cuadrícula (grid items).

## Propiedades del Grid Container

Estas propiedades se aplican al elemento padre que define la cuadrícula:

`display: grid / display: inline-grid` Define el contenedor como una cuadrícula.

`grid-template-columns` Especifica el tamaño y número de columnas.

`grid-template-rows` Define el tamaño y número de filas.

`grid-template-areas` Permite nombrar áreas dentro de la cuadrícula.

`grid-column-gap / column-gap` Espacio entre columnas.

`grid-row-gap / row-gap` Espacio entre filas.

`grid-gap / gap` Espacio entre filas y columnas.

`justify-items` Alinea los elementos dentro de sus celdas en el eje horizontal.

`align-items` Alinea los elementos dentro de sus celdas en el eje vertical.

`place-items` Combinación de align-items y justify-items.

`justify-content` Alinea la cuadrícula dentro del contenedor en el eje horizontal.

`align-content` Alinea la cuadrícula dentro del contenedor en el eje vertical.

`place-content` Combinación de align-content y justify-content.

`grid-auto-columns` Define el tamaño de las columnas generadas automáticamente.

`grid-auto-rows` Define el tamaño de las filas generadas automáticamente.

`grid-auto-flow` Controla cómo se colocan los elementos automáticamente en la cuadrícula.

### Propiedades del Grid Item

Estas propiedades se aplican a los elementos hijos dentro de la cuadrícula:

`grid-column-start` Define en qué línea de columna comienza el elemento.

`grid-column-end` Define en qué línea de columna termina el elemento.

`grid-column` Atajo para grid-column-start y grid-colu mn-end.

`grid-row-start` Define en qué línea de fila comienza el elemento.

`grid-row-end` Define en qué línea de fila termina el elemento.

`grid-row` Atajo para grid-row-start y grid-row-end.

`grid-area` Asigna un elemento a un área de la cuadrícula nombrada con grid-template-areas.

`justify-self` Alinea un elemento dentro de su celda en el eje horizontal.

`align-self` Alinea un elemento dentro de su celda en el eje vertical.

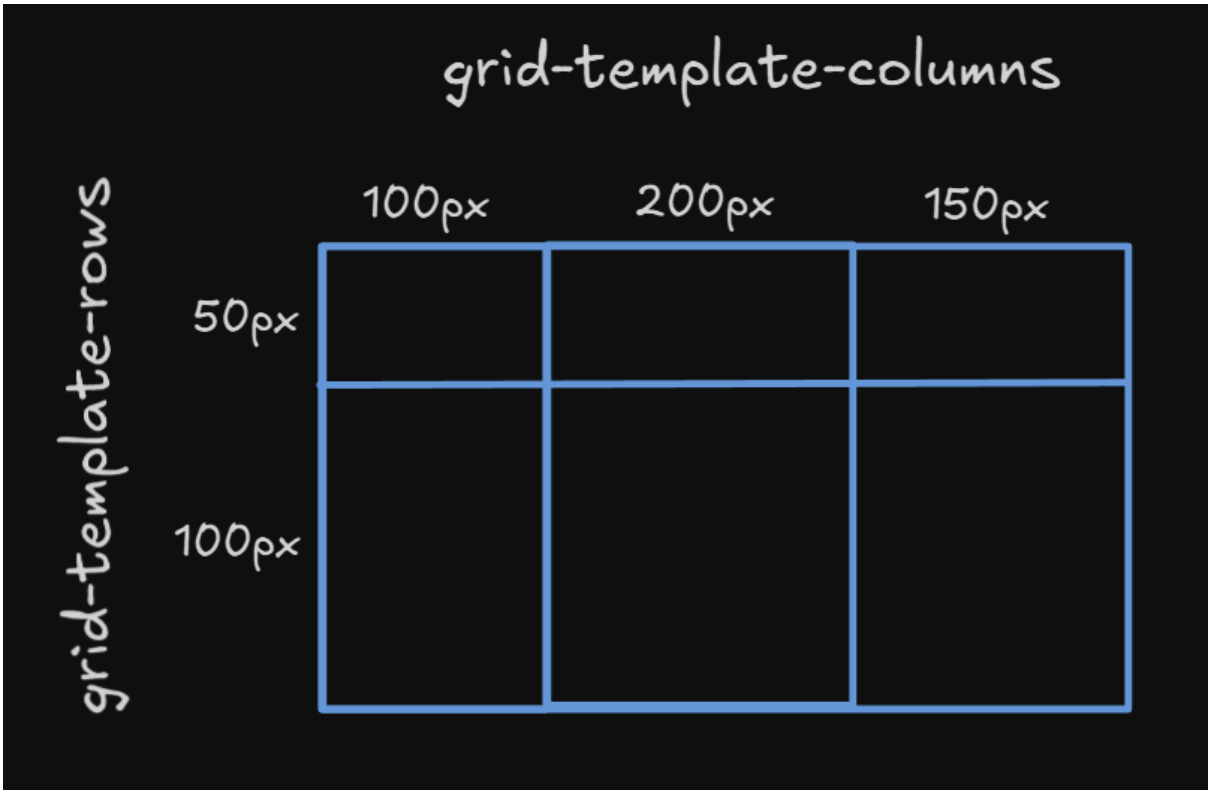
`place-self` Combinación de align-self y justify-self.

## Ejemplos prácticos

Crear una cuadrícula de 3 columnas y 2 filas:

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 150px;
  grid-template-rows: 50px 100px;
}
```

Con la propiedad `display: grid`, indicamos que el elemento será un contenedor de cuadrícula. Luego, con `grid-template-columns` y `grid-template-rows`, establecemos el tamaño y la cantidad de columnas y filas que compondrán la cuadrícula.




Es importante asegurarnos de que el número de elementos hijos dentro del grid sea el adecuado para la estructura que definimos. La cantidad de filas y columnas que configuremos con `grid-template-columns` y `grid-template-rows` determinará cómo se distribuyen los elementos.

Por ejemplo, si definimos una cuadrícula de 2x2, tendremos espacio para 4 elementos; una 2x3 podrá contener 6 elementos, una 2x4 albergará 8 elementos, y así sucesivamente.

Si el número de elementos no coincide con la cantidad de celdas disponibles, pueden ocurrir dos situaciones:

- **Si hay más elementos de los esperados,** los adicionales se colocarán automáticamente en nuevas filas sin un formato definido, a menos que establezcamos reglas específicas.
- **Si hay menos elementos,** solo se ocupan las celdas necesarias, y las restantes quedarán vacías.

A medida que añadimos más elementos al grid, podemos ajustar las propiedades `grid-template-columns` y `grid-template-rows` para expandir la estructura y acomodar el nuevo contenido de manera organizada.

### ¿Cómo funciona la unidad fr?

En el ejemplo anterior, hemos utilizado píxeles como unidad para definir el tamaño de las celdas de la cuadrícula. Sin embargo, CSS Grid nos permite usar otras unidades para lograr una mayor flexibilidad en el diseño. Algunas de las opciones más comunes incluyen:

- Porcentajes (%): Permiten definir columnas o filas en relación con el tamaño del contenedor padre.
- auto: Ajusta automáticamente el tamaño de la fila o columna según el contenido que contenga.
- fr (fracción restante): Una unidad especial de Grid que distribuye el espacio disponible de forma proporcional entre las columnas o filas.

La unidad `fr` representa una fracción del espacio disponible en la cuadrícula, es decir, el espacio que queda después de asignar tamaños fijos o automáticos. Veamos algunos ejemplos:

```
.container {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
}
```

Aquí estamos creando tres columnas:

- La primera columna ocupa 1 parte del espacio disponible.
- La segunda columna ocupa 2 partes (el doble que la primera y la tercera).
- La tercera columna ocupa 1 parte, igual que la primera.

Si el contenedor tuviera 600px de ancho, el cálculo sería:

- 1fr → 150px
- 2fr → 300px
- 1fr → 150px

La gran ventaja de `fr` es que nos permite distribuir el espacio de manera flexible, sin necesidad de cálculos manuales, adaptando la cuadrícula automáticamente al tamaño del contenedor.

### Repetir filas y columnas

Cuando necesitamos repetir el mismo tamaño en varias columnas o filas dentro de `grid-template-columns` o `grid-template-rows`, escribirlo manualmente puede volverse tedioso.

Para evitar esto, podemos utilizar la función `repeat()`, que nos permite definir cuántas veces queremos repetir un valor.

Ejemplo de repeat()

Supongamos que queremos una cuadrícula de cuatro columnas, cada una con un ancho de 100px. En lugar de escribirlo así:

```
grid-template-columns: 100px 100px 100px 100px;
```

Podemos simplificarlo con `repeat()` de la siguiente manera:

```
grid-template-columns: repeat(4, 100px);
```

Esto indica que queremos 4 columnas de 100px cada una.

También podemos combinar `repeat()` con otras unidades, por ejemplo:

```
grid-template-columns: repeat(3, 1fr) 200px;
```

Aquí definimos:

- Tres columnas flexibles (1fr cada una) que comparten el espacio disponible.
- Una cuarta columna fija de `200px`.

Esta función es muy útil cuando queremos definir estructuras dinámicas y escalables, especialmente en layouts responsivos.

## Función minmax()

La función `minmax()` nos permite definir un rango flexible para el tamaño de columnas o filas dentro de CSS Grid. Con esta función, establecemos un mínimo y un máximo para que el tamaño del elemento se ajuste dinámicamente según el espacio disponible.

Ejemplo de minmax()

Supongamos que queremos una cuadrícula con tres columnas, pero que cada columna tenga un tamaño mínimo de 150px y pueda crecer hasta llenar el espacio disponible. En lugar de usar valores fijos, podemos hacerlo así:

```
grid-template-columns: repeat(3, minmax(150px, 1fr));
```

¿Cómo funciona?

- 150px → Es el tamaño mínimo que puede tener cada columna.
- 1fr → Permite que las columnas crezcan proporcionalmente si hay espacio disponible.

Así, cuando la pantalla sea pequeña, las columnas nunca serán más estrechas de 150px, pero en pantallas más grandes, crecerán para ocupar el espacio restante de manera uniforme.

https://codepen.io/johnserranodev/pen/emYBaxY

### Otro caso de uso:

Si queremos una cuadrícula en la que una columna se mantenga fija, pero las demás sean flexibles, podemos combinar `minmax()` con valores fijos:

```
grid-template-columns: 200px minmax(150px, 1fr) minmax(150px, 1fr);
```

Aquí tenemos:

✅ Una columna fija de 200px. ✅ Dos columnas flexibles que nunca bajarán de 150px, pero pueden expandirse para ocupar el espacio disponible.

Esta función es extremadamente útil para layouts responsivos, evitando que las columnas o filas se vuelvan demasiado pequeñas o demasiado grandes.

### `auto-fill` y `auto-fit` en `repeat()`

Dentro de la función `repeat()`, podemos usar las palabras clave `auto-fill` y `auto-fit` para hacer que el navegador ajuste automáticamente el número de columnas en función del tamaño del contenedor y del `viewport`.

Esto nos permite crear cuadrículas dinámicas y responsivas sin necesidad de usar media queries manualmente.

¿Cómo funciona?

Si usamos:

```
grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
```

📌 El navegador calculará cuántas columnas de al menos 300px caben en el contenedor.

📌 Si hay más espacio, se agregarán más columnas automáticamente.

📌 Si no caben más columnas en la primera fila, los elementos restantes se moverán a una nueva fila.

Este comportamiento es similar a lo que hacemos con media queries, pero de manera automática y con menos código.

## Diferencia entre `auto-fill` y `auto-fit`

- `auto-fill` Rellena el espacio con columnas vacías si no hay suficientes elementos.
- `auto-fit` Ajusta el número de columnas sin dejar espacios vacíos.

Ejemplo con `auto-fill`

```
grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
```

Crea tantas columnas como quepan en el contenedor.

Si sobra espacio, se mantendrán las columnas vacías aunque no haya más ítems.

Ejemplo con `auto-fit`

```
grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
```

Crea tantas columnas como sea necesario.

Si hay menos elementos de los que caben, el espacio sobrante se redistribuye entre las columnas existentes.

Te dejo un ejemplo visual de `auto-git` y `auto-fill`.

https://codepen.io/johnserranodev/pen/vEYgyey

Si quieres ver correctamente la diferencia abrelo en una pestaña aparte: **Ejemplo práctico con 10 ítems**.

### Ejemplo práctico con 10 ítems

```
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 10px;
}

.item {
  background: #4CAF50;
  padding: 20px;
  text-align: center;
  color: white;
  font-size: 1.2em;
}
```

📌 Con `auto-fit`, los elementos ocuparán todo el espacio disponible.

📌 En pantallas pequeñas, el número de columnas se ajustará automáticamente.

📌 Si hay espacio extra, las columnas se expanden en lugar de dejar huecos vacíos.

Así, conseguimos un diseño responsivo sin necesidad de escribir múltiples reglas con media queries.

### Agregar espacios con gap

La propiedad `gap` en CSS Grid nos permite definir el espacio entre filas y columnas de manera sencilla, sin necesidad de usar `margin` o `padding` en los elementos hijos.

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
```

```
  gap: 20px; /* Espacio entre filas y columnas */
}
```

También podemos usar `row-gap` y `column-gap` si queremos definirlos por separado:

```
gap: 10px 20px; /* 10px entre filas, 20px entre columnas */
```

Esto hace que la cuadrícula sea más limpia y fácil de mantener.

## Grid por áreas en CSS: Una forma visual y flexible de diseñar cuadrículas

Hasta ahora, hemos visto cómo definir cuadrículas en CSS Grid utilizando `grid-template-rows` y `grid-template-columns`.

Sin embargo, existe una manera más intuitiva de organizar el diseño de la cuadrícula mediante Grid por áreas, lo que permite estructurar la disposición de los elementos de una forma más visual y flexible.

### ¿Qué es Grid por áreas?

El método de Grid por áreas nos permite asignar nombres específicos a diferentes secciones de la cuadrícula y organizarlas mediante la propiedad `grid-template-areas`.

Esto facilita la colocación de elementos sin necesidad de especificar manualmente sus posiciones con `grid-column` y `grid-row`.

💡 Importante:

Los grids por áreas no reemplazan la forma tradicional de definir cuadrículas mediante filas y columnas. Ambas técnicas pueden combinarse según las necesidades del diseño.

#### Propiedades clave en Grid por áreas

Para utilizar este método, debemos conocer dos propiedades esenciales:

- **grid-template-areas:** Define la disposición de las áreas en el grid. Cada texto entre comillas representa una fila.
- **grid-area:** Asigna un nombre específico a cada ítem hijo del grid.

### Ejemplo práctico de Grid por áreas

Veamos cómo podemos definir una cuadrícula con cabecera, menú lateral, contenido principal y pie de página de forma clara y ordenada:

```
.container {
  display: grid;
  grid-template-areas:
    "head head"
    "menu main"
    "foot foot";

  grid-template-columns: 1fr 2fr;
  grid-template-rows: 80px 200px 60px;
  gap: 10px;
}

.item-1 { grid-area: head; background: blue; }
.item-2 { grid-area: menu; background: red; }
.item-3 { grid-area: main; background: green; }
.item-4 { grid-area: foot; background: orange; }
```

`grid-template-areas` define el diseño en filas:

- La cabecera (head) ocupa toda la primera fila.
- El menú (menu) está a la izquierda y el contenido principal (main) a la derecha en la segunda fila.
- El pie de página (foot) ocupa toda la tercera fila.

`grid-area` asigna a cada elemento su nombre correspondiente.

`grid-template-columns` establece dos columnas: la primera ocupa 1fr (una fracción) y la segunda 2fr (el doble de la primera).

`grid-template-rows` define alturas fijas para las filas.

`gap` agrega espacio entre las celdas para mayor claridad visual.

### Valores Útiles en `grid-template-areas`

| Valor | Descripción |
| --- | --- |
| **none** | No define ninguna plantilla de áreas. |
| **”head”** | Una fila con un solo área llamada head. |
| **”head menu”** | Una fila con dos áreas (head y menu). |
| **“head head”** | Una fila con una única área que ocupa ambas columnas. |
| **”.”** | Define una celda vacía (sin nombre). |

⚠ Nota: Si asignamos un área en `grid-template-areas` pero no la definimos en `grid-area`, la propiedad será ignorada.

Te dejo un ejemplo para que puedas ver cómo funciona.

https://codepen.io/johnserranodev/pen/pvoRwmb

### Ejemplo completo de CSS Grid

Ya para terminar, veamos un ejemplo con todo lo aprendido:

```
<section class="container">
  <div class="item header">Header</div>
  <div class="item sidebar">Sidebar</div>
  <div class="item main">
    <h2>Main Content</h2>
    <ul class="list_option">
      <li class="option">option 1</li>
      <li class="option">option 2</li>
      <li class="option">option 3</li>
```

```
      <li class="option">option 4</li>
      <li class="option">option 5</li>
      <li class="option">option 6</li>
      <li class="option">option 7</li>
      <li class="option">option 8</li>
      <li class="option">option 9</li>
      <li class="option">option 10</li>
    </ul>
  </div>
  <div class="item footer">Footer</div>
</section>
```

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 columnas de igual tamaño */
  grid-template-rows: minmax(100px, auto) 800px 100px; /* Definimos el alto de
las filas */
  gap: 10px; /* Espaciado entre celdas */
  grid-template-areas:
    "header header header"
    "sidebar main main"
    "footer footer footer";
}

.item {
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 20px;
  font-weight: bold;
  color: white;
  padding: 20px;
}

.header { grid-area: header; background-color: #3498db; }
.sidebar { grid-area: sidebar; background-color: #e74c3c; }
.main {
  grid-area: main;
  background-color: #2ecc71;
  display: flex;
  flex-direction: column;
}
.footer { grid-area: footer; background-color: #f39c12; }


.list_option {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  padding: 0;
```

```
  gap: 10px;
  width: 100%;
  list-style: none;
}

.option {
  background: #000223;
  padding: 20px;
  text-align: center;
  color: white;
  font-size: 1em;
}

body {
  padding: 10px;
}
```

## Explicación:

> Se define un `grid-template-columns` con `repeat(3, 1fr)`, lo que genera tres columnas de igual tamaño.
> Se usa `minmax(100px, auto)` en la primera fila, permitiendo que crezca dinámicamente si el contenido lo requiere.
> El `gap`: 10px agrega un pequeño espacio entre las celdas.
> La propiedad `grid-template-areas` estructura visualmente la disposición del `grid`.
> Cada ítem tiene asignado un `grid-area` correspondiente (header, sidebar, main, footer).

Este código crea una cuadrícula con una cabecera arriba, un sidebar a la izquierda, un área de contenido principal en el centro y un pie de página abajo.

Cabe destacar que dentro de *Main Content* también tenemos un `grid` con `auto-fit`, que organiza automáticamente una lista de elementos según el espacio disponible.

Te dejo el ejemplo completo.

**https://codepen.io/johnserranodev/pen/yyLgXma**

## Conclusiones sobre CSS Grid

CSS Grid es una poderosa herramienta para diseñar layouts complejos de manera flexible y eficiente. A lo largo de este tema, hemos visto cómo estructurar cuadrículas utilizando `display: grid` y definir filas y columnas con `grid-template-columns` y `grid-template-rows`.

También exploramos conceptos clave como celdas, áreas, líneas y tracks, que nos permiten organizar mejor los elementos dentro del grid.